

Aggregation Commands Comparison

NOTE

Aggregation Pipeline as Alternative to Map-Reduce

Starting in MongoDB 5.0, map-reduce is deprecated:

- Instead of map-reduce, you should use an aggregation pipeline. Aggregation pipelines provide better performance and usability than map-reduce.
- You can rewrite map-reduce operations using aggregation pipeline stages, such as `$group`, `$merge`, and others.
- For map-reduce operations that require custom functionality, you can use the `$accumulator` and `$function` aggregation operators. You can use those operators to define custom aggregation expressions in JavaScript.

For examples of aggregation pipeline alternatives to map-reduce, see:

- Map-Reduce to Aggregation Pipeline
- Map-Reduce Examples

Aggregation Commands Comparison Table

The following table provides a brief overview of the features of the MongoDB aggregation commands.

	<code>aggregate</code> / <code>db.collection.aggregate()</code>	<code>mapReduce</code> / <code>db.collection.mapReduce()</code>
Description	Designed with specific goals of improving performance and usability for aggregation tasks. Uses a "pipeline" approach where objects are transformed as they pass through a series of pipeline operators such as <code>\$group</code>, <code>\$match</code>, and <code>\$sort</code>. See Aggregation Operators for more information on the pipeline operators.	Implements the Map-Reduce aggregation for processing large data sets.
Key Features	Pipeline operators can be repeated as needed. Pipeline operators need not produce one output document for every input document. Can also generate new documents or filter out documents. Using the <code>\$merge</code> stage, you can create on-demand materialized views, where the content of the output collection can be updated incrementally the pipeline is run. <code>\$merge</code> can incorporate results (insert new documents, merge documents, replace documents, keep	In addition to grouping operations, can perform complex aggregation tasks as well as perform incremental aggregation on continuously growing datasets. See Map-Reduce Examples and Perform Incremental Map-Reduce.

	existing documents, fail the operation, process documents with a custom update pipeline) into an existing collection.	
Flexibility	You can define custom aggregation expressions with <code>\$accumulator</code> and <code>\$function</code>. You can also add computed fields, create new virtual sub-objects, and extract sub-fields into the top-level of results by using the <code>\$project</code> pipeline operator. See <code>\$project</code> for more information as well as Aggregation Operators for more information on all the available pipeline operators.	Custom <code>map</code>, <code>reduce</code> and <code>finalize</code> JavaScript functions offer flexibility to aggregation logic. See <code>mapReduce</code> for details and restrictions on the functions.
Output Results	Returns results as a cursor. If the pipeline includes the <code>\$out</code> stage or <code>\$merge</code> stage, the cursor is empty. With <code>\$out</code>, you can replace an existing output collection completely or output to a new collection. See <code>\$out</code> for details. With <code>\$merge</code>, you can output to a new or existing collection. For existing collections, you can specify how to incorporate the results into the output collection (insert new documents, merge documents, replace documents, keep existing documents, fail the operation, process documents with a custom update pipeline). See <code>\$merge</code> for details.	Returns results in various options (inline, new collection, merge, replace, reduce). See <code>mapReduce</code> for details on the output options.
Sharding	Supports non-sharded and sharded input collections. <code>\$merge</code> can output to a non-sharded or sharded collection.	Supports non-sharded and sharded input collections.
More Information	- Aggregation Pipeline <code>db.collection.aggregate()</code> <code>aggregate</code>	- Map-Reduce <code>db.collection.mapReduce()</code> <code>mapReduce</code>.

TIP**See also:**

Map-Reduce to Aggregation Pipeline